

■ MongoDB & Express.js Notes

1■■ SQL vs NoSQL Databases

SQL (Relational Databases)

- Structured, table-based (rows & columns)
- Fixed schema, strict rules
- ACID transactions (reliable for banking)
- Vertical scaling (scale up hardware)
- Examples: MySQL, PostgreSQL, Oracle, MS SQL Server

NoSQL (Non-Relational Databases)

- Flexible, document/column/graph/key-value based
- Schema-less or dynamic schema
- BASE (eventual consistency)
- Horizontal scaling (scale out with more servers)
- Examples: MongoDB, Cassandra, Redis, Neo4j

2■■ Why MongoDB is Preferred in MERN Stack

- Stores data in JSON-like format (BSON), matches JavaScript objects
- Perfect integration with Node.js & React (no data conversion)
- Schema flexibility: add new fields easily
- Horizontally scalable: handles millions of users
- Great for rapid prototyping and startups
- Works smoothly with libraries like Mongoose

3■■ MongoDB Shell Commands

Database Commands:

- show dbs
- use myDB
- db
- db.dropDatabase()

Collection Commands:

- show collections
- db.createCollection("users")
- db.users.drop()

Insert Documents:

- db.users.insertOne({ name: "Alisha", age: 25 })
- db.users.insertMany([{ name: "Rahul", age: 30 }, { name: "Aisha", age: 22 }])

Read Documents:

- db.users.find()
- db.users.find().pretty()
- db.users.find({ name: "Alisha" })
- db.users.find({ age: { \$gt: 25 } })
- db.users.findOne({ name: "Rahul" })

Update Documents:

- db.users.updateOne({ name: "Alisha" }, { \$set: { age: 26 } })
- db.users.updateMany({ age: { \$lt: 25 } }, { \$set: { status: "young" } })

Delete Documents:

- db.users.deleteOne({ name: "Rahul" })
- db.users.deleteMany({ age: { \$lt: 23 } })

Other Useful Commands:

- db.users.countDocuments()
- db.users.find().limit(3)
- db.users.find().sort({ age: 1 })
- db.users.find().sort({ age: -1 })
- db.users.find({}, { name: 1, age: 1 })

4■■■ Node.js + Express + MongoDB (Without Mongoose)

Install dependencies:

```
npm init -y
npm install express mongodb dotenv
```

```
.env
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017
DB_NAME=myLocalDB

server.js
-----
const express = require("express");
const { MongoClient } = require("mongodb");
require("dotenv").config();

const app = express();
app.use(express.json());

let db;

// Connect to MongoDB
MongoClient.connect(process.env.MONGO_URI, { useUnifiedTopology: true })
  .then(client => {
    console.log("■ Connected to Local MongoDB");
    db = client.db(process.env.DB_NAME);
  })
  .catch(err => console.error("■ Connection Error:", err));

// Test Route
app.get("/", (req, res) => {
  res.send("Hello from Express + Local MongoDB (No Mongoose!)");
});

// Insert User
app.post("/users", async (req, res) => {
  try {
    const result = await db.collection("users").insertOne(req.body);
    res.json(result);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Get All Users
app.get("/users", async (req, res) => {
  try {
    const users = await db.collection("users").find().toArray();
    res.json(users);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`■ Server running on port ${PORT}`));
```

5■■■ Node.js + Express + MongoDB (With Mongoose)

Install dependencies:

```
npm init -y
npm install express mongoose dotenv
```

```
.env
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/myLocalDB
```

```
server.js
-----
const express = require("express");
const mongoose = require("mongoose");
require("dotenv").config();
const app = express();
```

```

app.use(express.json());

// MongoDB Connection (Local)
mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log("■ Connected to Local MongoDB"))
.catch(err => console.error("■ Connection Error:", err));

// User Model
const User = mongoose.model("User", new mongoose.Schema({
  name: String,
  email: String,
  age: Number
}));

// Insert User
app.post("/users", async (req, res) => {
  try {
    const user = new User(req.body);
    const savedUser = await user.save();
    res.json(savedUser);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Get All Users
app.get("/users", async (req, res) => {
  const users = await User.find();
  res.json(users);
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`■ Server running on port ${PORT}`));

```